PHÂN HIỆU TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
TẠI TP. HỒ CHÍ MINH

BỘ MÔN CÔNG NGHỆ THÔNG TIN

----ꕥ📖ꕥ----

KHAI PHÁ DỮ LIỆU

BÁO CÁO THỰC HÀNH BUỔI 1

<Họ tên sinh viên> - <MSSV>

<Lớp>

GVHD: Ths. Nguyễn Thiện Dương

TP. Hồ Chí Minh - 2025

Lưu ý:

1. *Số trang trong tài liệu thực hành tính theo số trang ứng dụng đọc PDF đếm*
2. *SV bố trí mỗi câu ở dưới sẽ là 1 file mã nguồn riêng (file.ipynb)*
3. *Những câu nào có yêu cầu lập trình viết mã nguồn thì SV phải in ra họ tên – MSSV (lệnh print) trong câu đó*
4. ***Hình chụp kết quả phải có chứa thanh taskbar của máy tính***
5. *SV đặt tên cho file mã nguồn mỗi câu như sau:* ***<MSSV>_LAB1_Bai<X>***
6. *Tất cả các câu sau đó sẽ tổng hợp lại và push lên 1 repository duy nhất trên Github*
7. *Đặt tên cho repository theo cú pháp:* ***<MSSV>_Lab1***

BÁO CÁO PHẦN 1

https://github.com/legialuan780/6451071045_Lab1.git

Bài 2 (Trang 15-17)

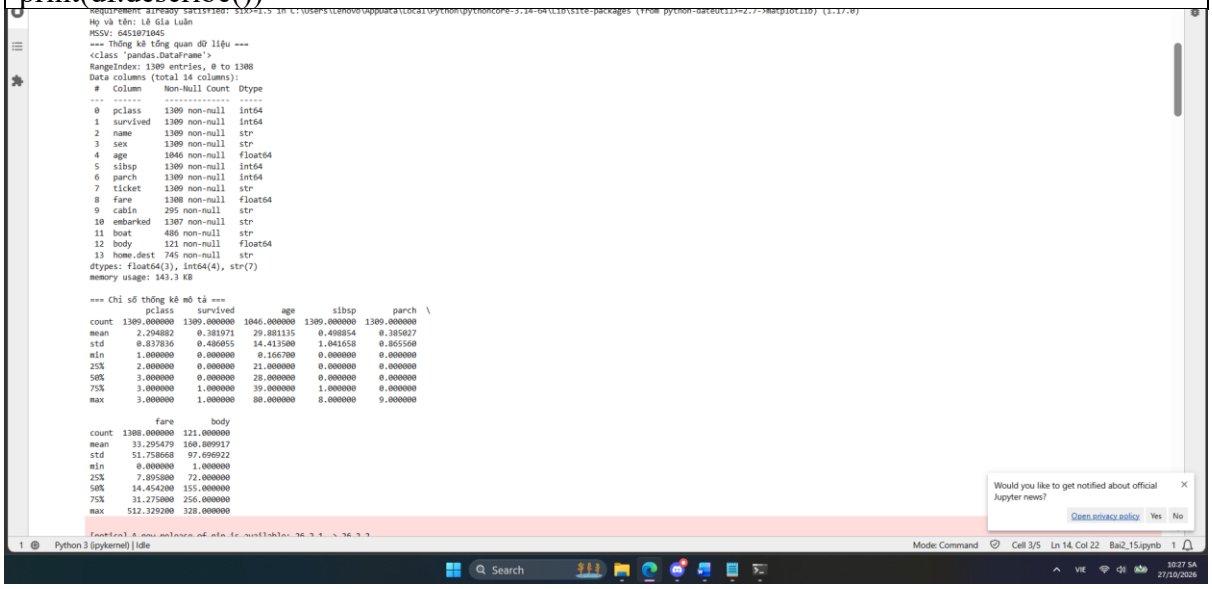
Câu 1:

```
# Khai báo thông tin sinh viên
MSSV = "6451071045"
HO_TEN = "Lê Gia Luân"

print(f"Họ và tên: {HO_TEN}")
print(f"MSSV: {MSSV}")
!pip install matplotlib seaborn
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import re

# 1. Nhập dữ liệu và thống kê thông tin cơ bản
df = pd.read_csv('titanic.csv', dtype={'age': np.float64, 'sibsp': np.int64, 'parch': np.int64})

print("=== Thống kê tổng quan dữ liệu ===")
df.info()
print("\n=== Chỉ số thống kê mô tả ===")
print(df.describe())
```

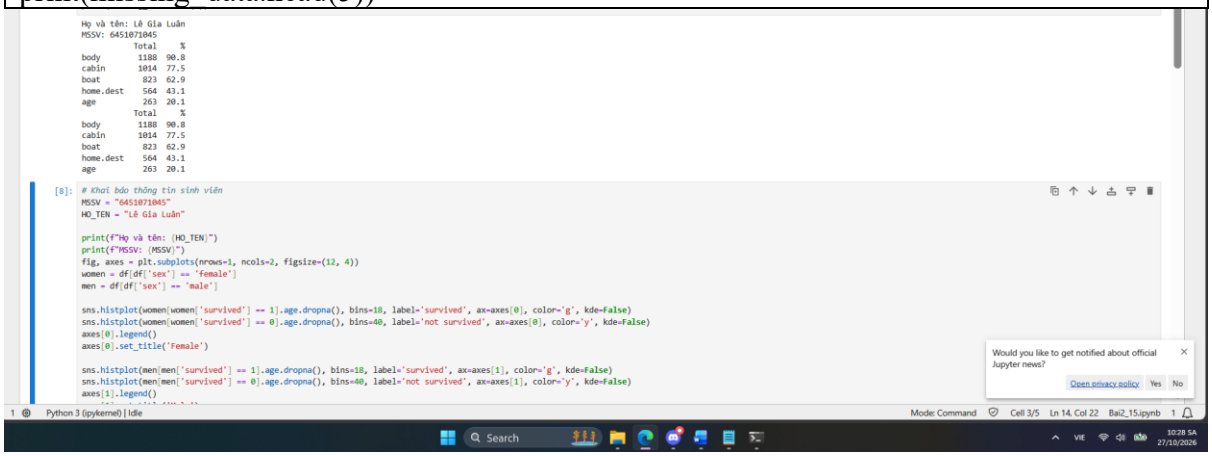


Câu 2:

```
# Khai báo thông tin sinh viên
MSSV = "6451071045"
```

```
HO_TEN = "Lê Gia Luân"

print(f"Họ và tên: {HO_TEN}")
print(f"MSSV: {MSSV}")
total = df.isnull().sum().sort_values(ascending=False)
percent = (round(df.isnull().sum() / df.isnull().count() * 100,
1)).sort_values(ascending=False)
missing_data = pd.concat([total, percent], axis=1, keys=['Total', '%'])
print(missing_data.head(5))
```



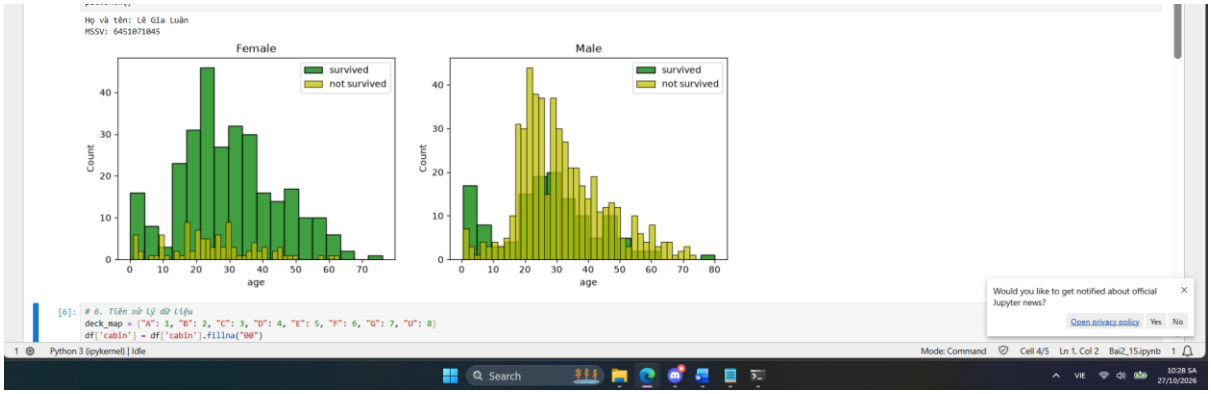
Câu 3:

```
# Khai báo thông tin sinh viên
MSSV = "6451071045"
HO_TEN = "Lê Gia Luân"

print(f"Họ và tên: {HO_TEN}")
print(f"MSSV: {MSSV}")
fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(12, 4))
women = df[df['sex'] == 'female']
men = df[df['sex'] == 'male']

sns.histplot(women[women['survived'] == 1].age.dropna(), bins=18, label='survived',
ax=axes[0], color='g', kde=False)
sns.histplot(women[women['survived'] == 0].age.dropna(), bins=40, label='not survived',
ax=axes[0], color='y', kde=False)
axes[0].legend()
axes[0].set_title('Female')

sns.histplot(men[men['survived'] == 1].age.dropna(), bins=18, label='survived', ax=axes[1],
color='g', kde=False)
sns.histplot(men[men['survived'] == 0].age.dropna(), bins=40, label='not survived',
ax=axes[1], color='y', kde=False)
axes[1].legend()
axes[1].set_title('Male')
plt.show()
```



Nhận xét:

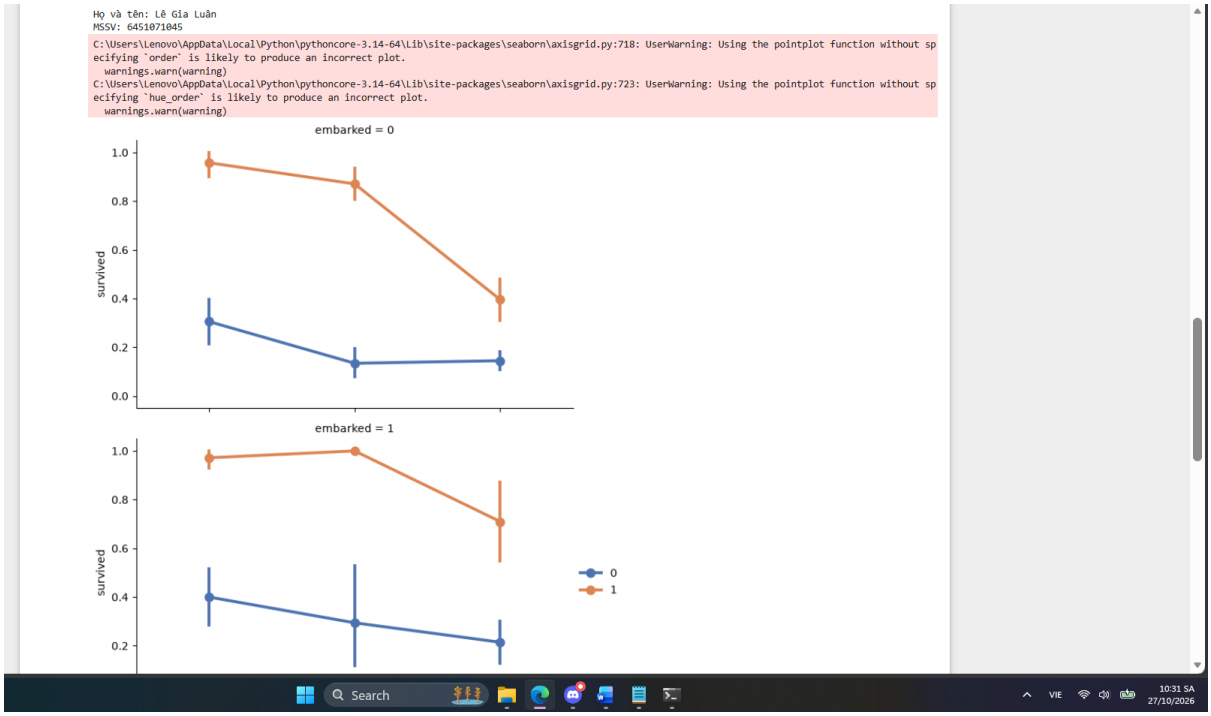
- Nam giới có xác suất sống sót cao nhất ở độ tuổi trẻ em (1 đến 10 tuổi).
- Nữ giới có tỷ lệ sống sót cao vượt trội hơn nam giới ở hầu hết mọi độ tuổi, đặc biệt trong khoảng từ 12 đến 40 tuổi.
- Tỷ lệ nam giới không sống sót tập trung cao nhất ở độ tuổi từ 18 đến 27 tuổi.

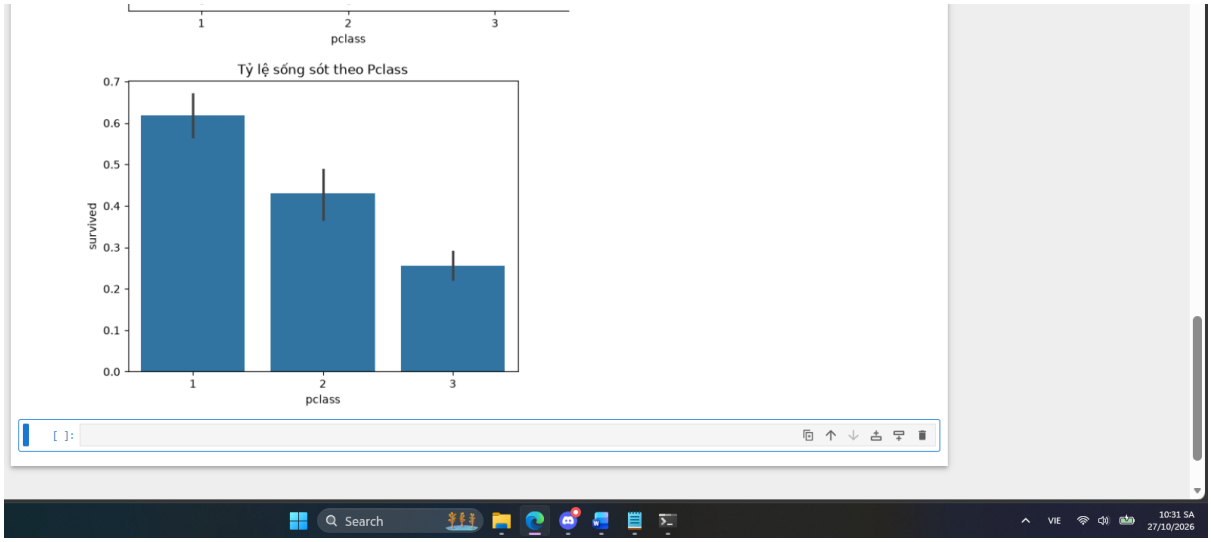
Câu 4:

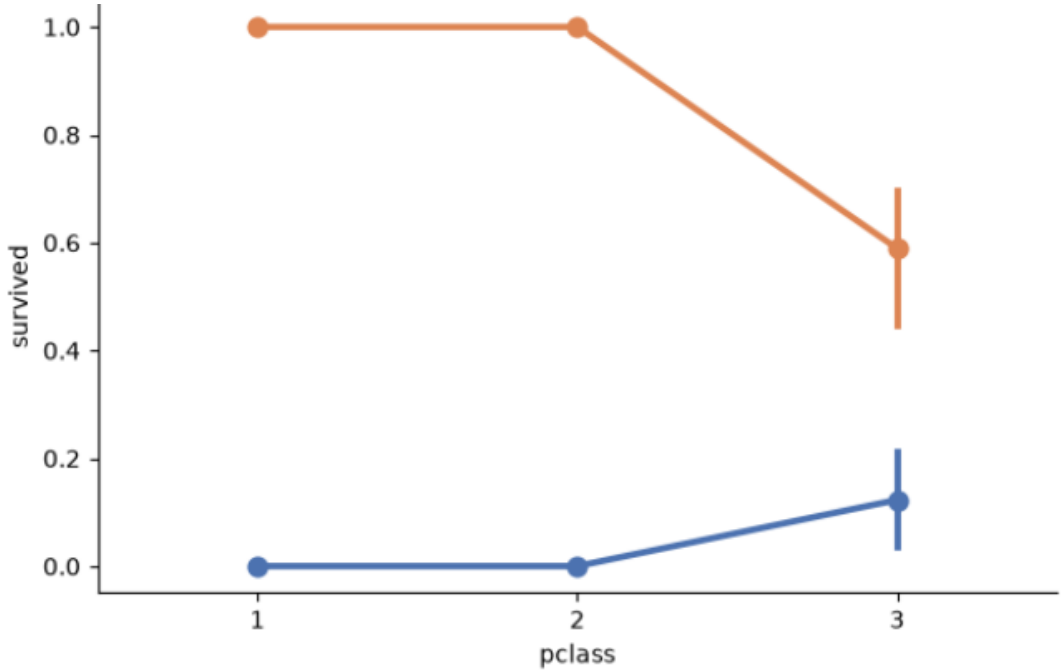
```
# Khai báo thông tin sinh viên
MSSV = "6451071045"
HO_TEN = "Lê Gia Luân"

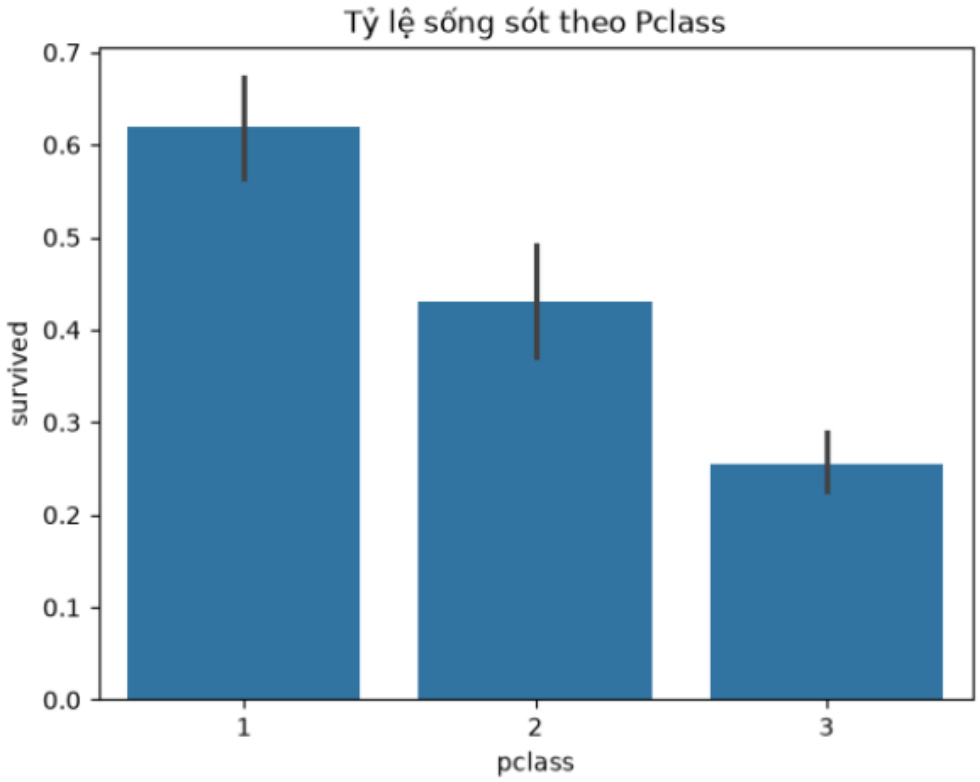
print(f"Họ và tên: {HO_TEN}")
print(f"MSSV: {MSSV}")
# Cổng lên tàu & Loại vé phân biệt theo Giới tính
grid = sns.FacetGrid(df, row='embarked', height=4, aspect=1.6)
grid.map(sns.pointplot, 'pclass', 'survived', 'sex', palette='deep')
grid.add_legend()
plt.show()

# Biểu đồ cột thể hiện tỷ lệ sống sót theo Loại vé
sns.barplot(x='pclass', y='survived', data=df)
plt.title("Tỷ lệ sống sót theo Pclass")
plt.show()
```


Họ và tên: Lê Gia Luân
MSSV: 6451071045
C:\Users\Lenovo\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\seaborn\axisgrid.py:718: UserWarning: Using the pointplot function without specifying `order` is likely to produce an incorrect plot.
warnings.warn(warning)
C:\Users\Lenovo\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\seaborn\axisgrid.py:723: UserWarning: Using the pointplot function without specifying `hue_order` is likely to produce an incorrect plot.
warnings.warn(warning)
embarked = 0
embarked = 1
survived
0
1
Search


pclass
Tỷ lệ sống sót theo Pclass
survived
pclass
[ ]:
Search





Nhận xét:

- Hành khách nữ lên tàu ở cổng C và Q có tỷ lệ sống sót gần như tuyệt đối đối với vé hạng 1 và 2.
- Hành khách sở hữu loại vé hạng 1 (pclass = 1) có cơ hội sống sót cao nhất (>60%), trong khi hành khách sở hữu vé hạng 3 (pclass = 3) có tỷ lệ tử vong cao nhất.

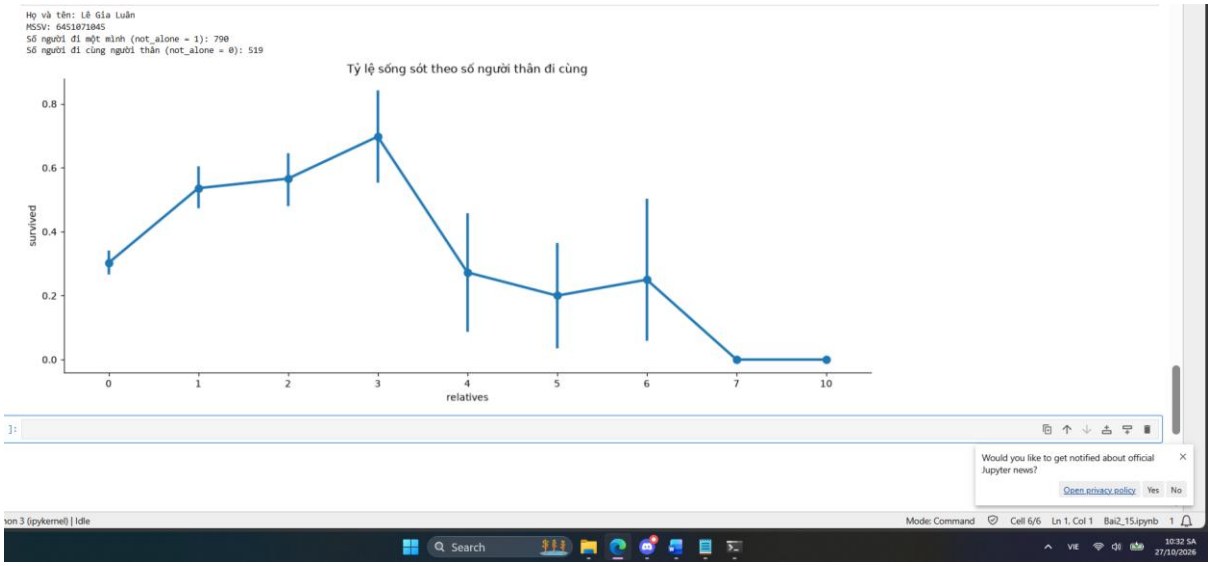
Câu 5:

```
# Khai báo thông tin sinh viên
MSSV = "6451071045"
HO_TEN = "Lê Gia Luân"

print(f"Họ và tên: {HO_TEN}")
print(f"MSSV: {MSSV}")
df['relatives'] = df['sibsp'] + df['parch']
df['not_alone'] = (df['relatives'] == 0).astype(int)

print("Số người đi một mình (not_alone = 1):", df['not_alone'].value_counts()[1])
print("Số người đi cùng người thân (not_alone = 0):", df['not_alone'].value_counts()[0])

sns.catplot(x='relatives', y='survived', data=df, aspect=2.5, kind='point')
plt.title("Tỷ lệ sống sót theo số người thân đi cùng")
plt.show()
```



Nhận xét:

- Những hành khách đi cùng từ 1 đến 3 người thân có tỷ lệ sống sót cao nhất (đạt khoảng 50%–70%).
- Hành khách đi một mình hoặc đi cùng gia đình quá đông (từ 7 đến 10 người) có tỷ lệ sống sót rất thấp.

Bài 2 (Trang 28)

Câu 1:

```
import pandas as pd
```

```
import numpy as np
# Khai báo thông tin sinh viên
MSSV = "6451071045"
HO_TEN = "Lê Gia Luân"

print(f"Họ và tên: {HO_TEN}")
print(f"MSSV: {MSSV}")
# Đọc dữ liệu Bank Marketing
df_bank = pd.read_csv('bank.csv', sep=';')

# 1. Xem thông tin cơ bản
print("=== Thông tin bộ dữ liệu Bank Marketing ===")
df_bank.info()

# 2. Kiểm tra các giá trị 'unknown' (thiếu thông tin)
unknown_counts = (df_bank == 'unknown').sum()
print("\n=== Số lượng giá trị 'unknown' theo từng thuộc tính ===")
print(unknown_counts[unknown_counts > 0])

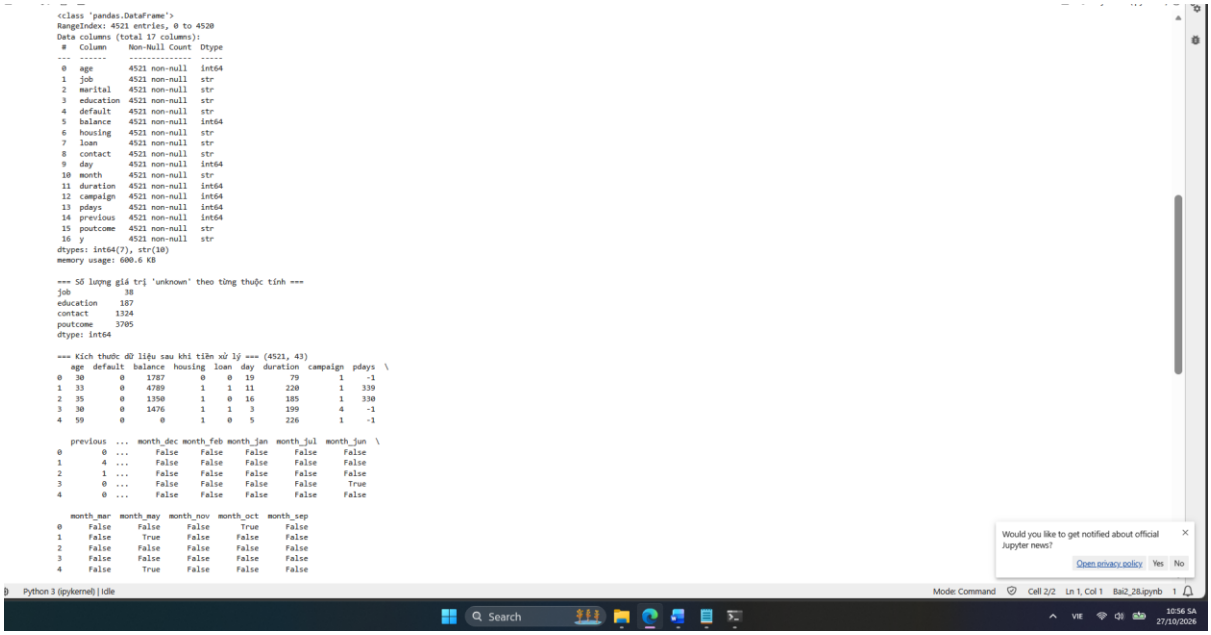
# 3. Mã hóa thuộc tính nhị phân (Binary Encoding)
binary_cols = ['default', 'housing', 'loan', 'y']
for col in binary_cols:
    df_bank[col] = df_bank[col].map({'yes': 1, 'no': 0})

# 4. Tiền xử lý dữ liệu 'unknown'
# Điền Mốt cho 'job' và 'education'
df_bank['job'] = df_bank['job'].replace('unknown', df_bank['job'].mode()[0])
df_bank['education'] = df_bank['education'].replace('unknown',
df_bank['education'].mode()[0])

# 5. Chia nhóm các thuộc tính định lượng liên tục (Binning)
df_bank['age_group'] = pd.qcut(df_bank['age'], q=5, labels=[0, 1, 2, 3, 4])
df_bank['duration_group'] = pd.qcut(df_bank['duration'], q=5, labels=[0, 1, 2, 3, 4])

# 6. Biến đổi thuộc tính dạng Categorical bằng One-Hot Encoding
df_bank_encoded = pd.get_dummies(
    df_bank,
    columns=['job', 'marital', 'education', 'contact', 'poutcome', 'month'],
    drop_first=True
)

print("\n=== Kích thước dữ liệu sau khi tiền xử lý ===", df_bank_encoded.shape)
print(df_bank_encoded.head())
```



Bài 2(Trang 30)

```
# Khai báo thông tin sinh viên
MSSV = "6451071045"
HO_TEN = "Lê Gia Luân"

print(f"Họ và tên: {HO_TEN}")
print(f"MSSV: {MSSV}")
import pandas as pd
import matplotlib.pyplot as plt

# a) Đọc file và đặt lại tiêu đề các cột
df_tb = pd.read_excel('tb_tracking.xlsx', header=None)
df_tb.columns = ['id', 'device_id', 'co_level', 'humidity', 'temperature', 'time']

# Chuyển cột time về định dạng datetime
df_tb['time'] = pd.to_datetime(df_tb['time'])

# b) Lọc bỏ dữ liệu trước ngày 03/01/2018
df_filtered = df_tb[df_tb['time'] >= '2018-01-03'].copy()

# c) Bóc tách thời điểm đo đạc trong ngày tính theo phút (minutes)
df_filtered['minutes'] = (
    df_filtered['time'].dt.hour * 60 +
    df_filtered['time'].dt.minute +
    df_filtered['time'].dt.second / 60.0
)

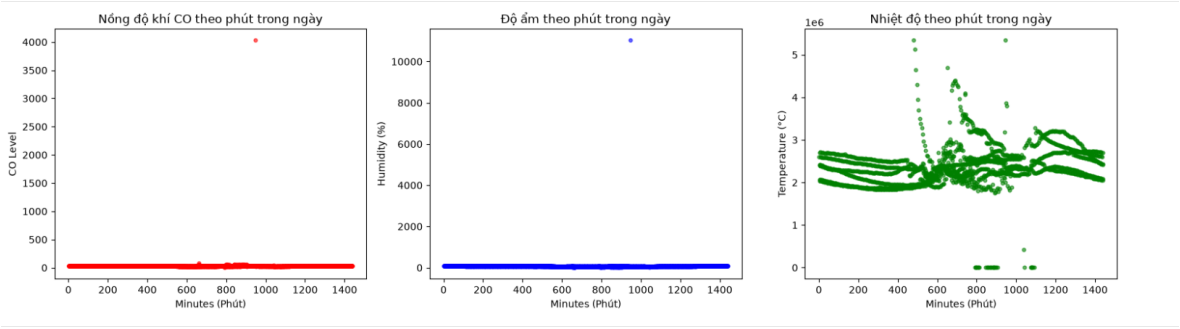
# d) Vẽ đồ thị phân tán (scatter plots)
fig, axes = plt.subplots(1, 3, figsize=(16, 4.5))
```

```
# Đồ thị Nồng độ CO
axes[0].scatter(df_filtered['minutes'], df_filtered['co_level'], alpha=0.6, color='red', s=12)
axes[0].set_title('Nồng độ khí CO theo phút trong ngày')
axes[0].set_xlabel('Minutes (Phút)')
axes[0].set_ylabel('CO Level')

# Đồ thị Độ ẩm
axes[1].scatter(df_filtered['minutes'], df_filtered['humidity'], alpha=0.6, color='blue', s=12)
axes[1].set_title('Độ ẩm theo phút trong ngày')
axes[1].set_xlabel('Minutes (Phút)')
axes[1].set_ylabel('Humidity (%)')

# Đồ thị Nhiệt độ
axes[2].scatter(df_filtered['minutes'], df_filtered['temperature'], alpha=0.6, color='green',
s=12)
axes[2].set_title('Nhiệt độ theo phút trong ngày')
axes[2].set_xlabel('Minutes (Phút)')
axes[2].set_ylabel('Temperature (°C)')

plt.tight_layout()
plt.show()
```



Lưu ý:

8. *Số trang trong tài liệu thực hành tính theo số trang ứng dụng đọc PDF đếm*
9. *Những câu nào có yêu cầu lập trình viết mã nguồn thì SV phải in ra họ tên – MSSV (lệnh print) trong câu đó*

BÁO CÁO PHẦN 2

https://github.com/legialuan780/6451071045_Lab1.git

Bài 2 (Trang 37, 38)

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from mlxtend.frequent_patterns import apriori, fpgrowth, association_rules
# Khai báo thông tin sinh viên
MSSV = "6451071045"
HO_TEN = "Lê Gia Luân"

# 1. Đọc dữ liệu từ file Excel
df_retail = pd.read_excel('Online Retail.xlsx', engine='openpyxl')

# 2. Tiền xử lý dữ liệu: xóa khoảng trắng thừa ở tên sản phẩm, xóa dòng rỗng ở InvoiceNo
df_retail['Description'] = df_retail['Description'].str.strip()
df_retail.dropna(subset=['InvoiceNo'], inplace=True)
df_retail['InvoiceNo'] = df_retail['InvoiceNo'].astype('str')

# 3. Loại bỏ các hóa đơn tín dụng (chứa ký tự 'C')
df_retail = df_retail[~df_retail['InvoiceNo'].str.contains('C', na=False)]

# 4 & 5. Lọc dữ liệu 'United Kingdom' và gom nhóm số lượng theo InvoiceNo và
Description
basket = (
    df_retail[df_retail['Country'] == 'United Kingdom']
    .groupby(['InvoiceNo', 'Description'])['Quantity']
    .sum().unstack().reset_index().fillna(0)
    .set_index('InvoiceNo')
)

# 6 & 7. Chuyển dữ liệu sang dạng One-Hot Encoding (0 và 1)
def encode_units(x):
    return 1 if x >= 1 else 0

# Sử dụng map thay cho applymap để tương thích với Pandas mới
basket_sets = basket.map(encode_units)

# 8. Xóa cột POSTAGE nếu có trong dữ liệu
if 'POSTAGE' in basket_sets.columns:
    basket_sets.drop('POSTAGE', inplace=True, axis=1)

# 9. Tìm tập phổ biến bằng thuật toán Apriori với min_sup = 3%
frequent_itemsets_apriori = apriori(basket_sets, min_support=0.03, use_colnames=True)

# 10. Tạo luật kết hợp với min_conf = 50% (0.50)
rules_apriori = association_rules(frequent_itemsets_apriori, metric="confidence",
min_threshold=0.50)

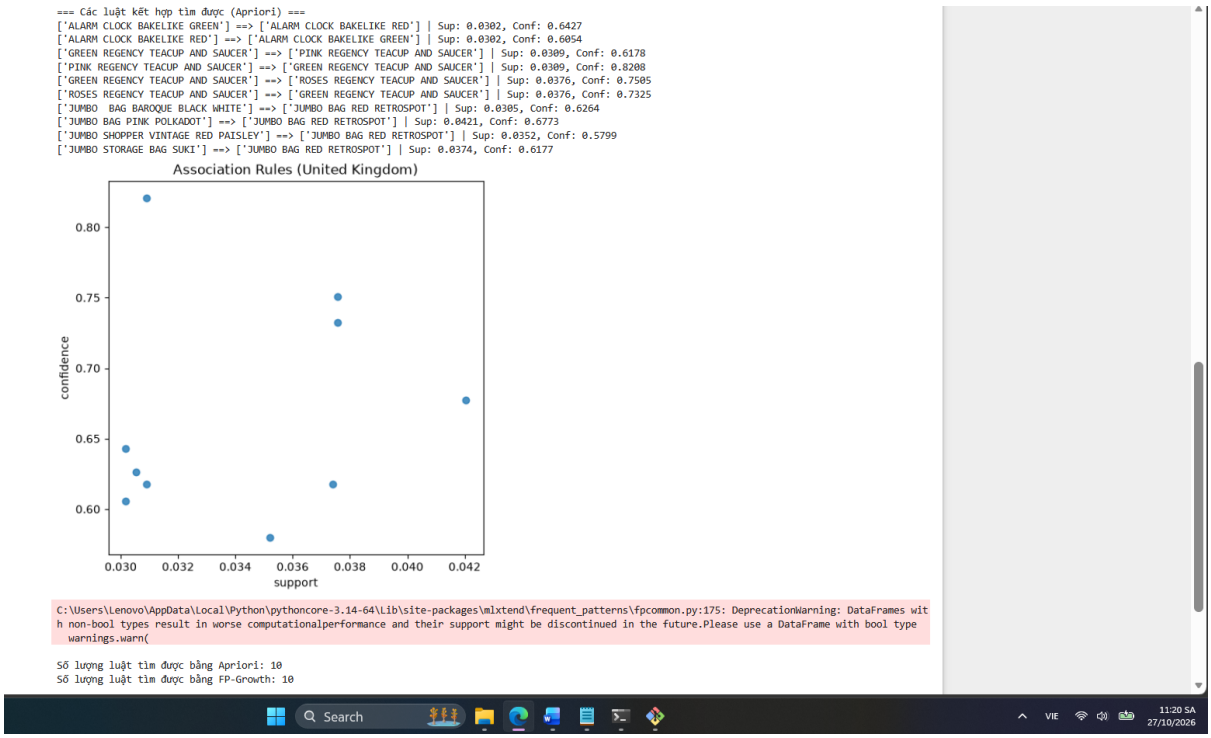
print("=== Các luật kết hợp tìm được (Apriori) ===")
```

```
for idx, row in rules_apriori.iterrows():
    print(f"{list(row['antecedents'])} ==> {list(row['consequents'])} | Sup:
{row['support']:.4f}, Conf: {row['confidence']:.4f}")

# 11. Vẽ đồ thị scatter plot biểu diễn Support và Confidence
plt.figure(figsize=(6, 6))
plt.title('Association Rules (United Kingdom)')
plt.xlabel('Support')
plt.ylabel('Confidence')
sns.regplot(x=rules_apriori['support'], y=rules_apriori['confidence'], fit_reg=False)
plt.show()

# 12. Tìm tập phổ biến & luật kết hợp bằng thuật toán FP-Growth
frequent_itemsets_fp = fpgrowth(basket_sets, min_support=0.03, use_colnames=True)
rules_fp = association_rules(frequent_itemsets_fp, metric="confidence",
min_threshold=0.50)

print(f"\nSố lượng luật tìm được bằng Apriori: {len(rules_apriori)}")
print(f"Số lượng luật tìm được bằng FP-Growth: {len(rules_fp)}")
```



Bài 5a (Trang 45)

Câu 1:

```
import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules
```

```
# Khai báo thông tin sinh viên
MSSV = "6451071045"
HO_TEN = "Lê Gia Luân"

# 1. Đọc file dữ liệu Online Retail (Nên đảm bảo đã cài openpyxl và mlxtend)
df_retail = pd.read_excel('Online Retail.xlsx', engine='openpyxl')

# 2. Tiền xử lý dữ liệu cơ bản
df_retail['Description'] = df_retail['Description'].str.strip()
df_retail.dropna(subset=['InvoiceNo'], inplace=True)
df_retail['InvoiceNo'] = df_retail['InvoiceNo'].astype('str')

# 3. Loại bỏ hóa đơn tín dụng (chứa ký tự 'C')
df_retail = df_retail[~df_retail['InvoiceNo'].str.contains('C', na=False)]

# 4. Lọc dữ liệu nước Đức ('Germany') và gom nhóm theo InvoiceNo và Description
basket_germany = (
    df_retail[df_retail['Country'] == 'Germany']
    .groupby(['InvoiceNo', 'Description'])['Quantity']
    .sum().unstack().reset_index().fillna(0)
    .set_index('InvoiceNo')
)

# 5. Chuyển đổi dữ liệu sang dạng One-Hot Encoding (0 và 1)
def encode_units(x):
    return 1 if x >= 1 else 0

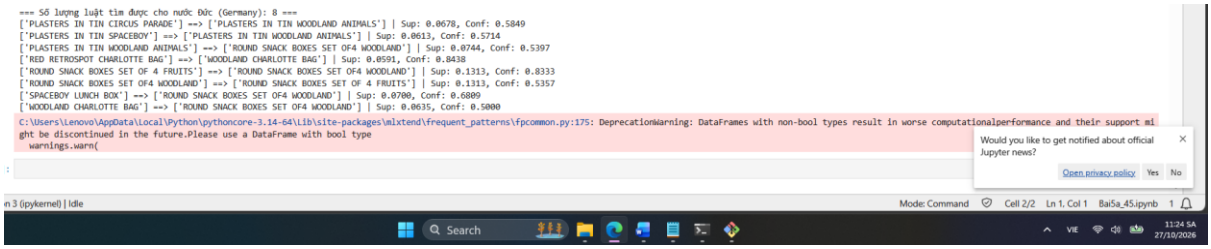
basket_germany = basket_germany.map(encode_units)

# 6. Xóa cột 'POSTAGE' nếu có
if 'POSTAGE' in basket_germany.columns:
    basket_germany.drop('POSTAGE', inplace=True, axis=1)

# 7. Tìm tập phổ biến bằng thuật toán Apriori với min_sup = 5% (0.05)
frequent_itemsets_germany = apriori(basket_germany, min_support=0.05,
use_colnames=True)

# 8. Tạo các luật kết hợp với min_conf = 50% (0.50)
rules_germany = association_rules(frequent_itemsets_germany, metric="confidence",
min_threshold=0.50)

# 9. In kết quả các luật kết hợp tìm được
print(f"=== Số lượng luật tìm được cho nước Đức (Germany): {len(rules_germany)}
===")
for idx, row in rules_germany.iterrows():
    print(f"{list(row['antecedents'])} ==> {list(row['consequents'])} | Sup:
{row['support']:.4f}, Conf: {row['confidence']:.4f}")
```



Bài 6 (Trang 45)

```
import time
import pandas as pd
import matplotlib.pyplot as plt
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, fpgrowth
# Khai báo thông tin sinh viên
MSSV = "6451071045"
HO_TEN = "Lê Gia Luân"

# 1. Đọc dữ liệu dạng danh sách các giao dịch (groceries.csv)
transactions = []
with open('groceries.csv', 'r') as f:
    for line in f:
        transaction = [item.strip() for item in line.strip().split(',') if item.strip()]
        if transaction:
            transactions.append(transaction)

# 2. Biến đổi dữ liệu thành One-Hot Encoding bằng TransactionEncoder
te = TransactionEncoder()
te_ary = te.fit(transactions).transform(transactions)
df_groceries = pd.DataFrame(te_ary, columns=te.columns_)

# 3. Các ngưỡng min_sup cần khảo sát (1% đến 10%)
min_sup_list = [0.01, 0.02, 0.03, 0.04, 0.05, 0.06, 0.08, 0.10]
apriori_times = []
fpgrowth_times = []

for sup in min_sup_list:
    # Đo thời gian Apriori
    t0 = time.time()
    apriori(df_groceries, min_support=sup, use_colnames=True)
    t1 = time.time()
    apriori_times.append(t1 - t0)

    # Đo thời gian FP-Growth
    t0 = time.time()
    fpgrowth(df_groceries, min_support=sup, use_colnames=True)
    t1 = time.time()
    fpgrowth_times.append(t1 - t0)

# c) Vẽ biểu đồ so sánh thời gian xử lý
```

```
plt.figure(figsize=(10, 5))
plt.plot(min_sup_list, apriori_times, marker='o', label='Apriori', color='red')
plt.plot(min_sup_list, fpgrowth_times, marker='s', label='FP-Growth', color='blue')
plt.title('So sánh thời gian xử lý giữa Apriori và FP-Growth')
plt.xlabel('Minimum Support (min_sup)')
plt.ylabel('Thời gian thực thi (giây)')
plt.legend()
plt.grid(True)
plt.show()
```

